Beginner's Guide to ESGF

This guide is targetted at users who are new to obtaining CMIP data from ESGF. While many people work hard to provide the community access in an intuitive fashion, ESGF remains a data source for researchers who have some prior understanding about the data they wish to find and how they are organized. This tutorial is meant to gently expose the uninitiated to key concepts and step you through your first searches using `intake-esgf`.

Which Variable Do We Need?

At the highest level, ESGF stores data in *projects* such as `CMIP5` and `CMIP6`. While there are some similarities between projects, the *control vocabulary*, that is the metadata used to identify unique datasets, varies. In this tutorial we will explain some of the CMIP6 vocabulary, which is the default project for `intake-esgf`.

Perhaps the most important search criteria to determine is the name of the variable you wish to use. `intake-esgf` has some functionality to assist. First, import and instantiate the catalog.

```
from intake_esgf import ESGFCatalog
cat = ESGFCatalog()
```

Then you can use the catalog to perform a free text search for any word that may be related to the variable for which you are searching. In this case, we will search for `air temperature surface`.

```
cat.variable_info("air temperature surface")
```

variable_id	cf_standard_name	variable_units	variable_long_name
hfls	surface_upward_latent_heat_flux	W m-2	Surface Upward Latent Heat Flux
hfss	surface_upward_sensible_heat_flux	W m-2	Surface Upward Sensible Heat Flux
rlds	surface_downwelling_longwave_flux_in_air	W m-2	Surface Downwelling Longwave Radiation
rsds	surface_downwelling_shortwave_flux_in_air	W m-2	Surface Downwelling Shortwave Radiation
sfcWind	wind_speed	m s-1	Near-Surface Wind Speed
ta	air_temperature	K	Air Temperature
tas	air_temperature	K	Near-Surface Air Temperature
tasmin	air_temperature	K	Daily Minimum Near-Surface Air Temperature
uas	eastward_wind	m s-1	Eastward Near-Surface Wind
vas	northward_wind	m s-1	Northward Near-Surface Wind

This function returns a pandas dataframe which lists the name of several variables along with their units and standard names. From a perusal of this list, it appears that `tas` is the variable we want for our search. The dataframe index also shows us that the name of the control vocabulary is `variable_id`.

Control Vocabulary

While we could now perform a search for `variable_id=tas`, this search will take quite some time. `intake-esgf` currently works better if we give it a better idea of what we wish to find. Simply put, we recommend constraining the search.

One of the more useful search facets is the `experiment_id`, a unique identifier corresponding to the experiment. As part of the planning phase of the CMIP process, groups of researchers write papers detailing the specific method that a model is to be run to be included in an experiment. This allows modeling centers to follow the protocol if they wish to be part of the experiment. You can browse the experiments to see the indentifiers and some basic information.

One commonly used experiment is `historical`, where models are run using reconstructions of the historical earth state from 1850 until 2015. We will use this in our example search.

```
cat.search(variable_id="tas",experiment_id="historical")
```

```
Searching indices: 100%                                  2/2 [      1.58index/s]
```

```
Summary information for 1686 results:
mip_era                                                  [CMIP6]
activity_drs                                              [CMIP]
institution_id    [NCAR, MIROC, NUIST, EC-Earth-Consortium, IPSL...
source_id         [CESM2-FV2, MIROC-ES2L, NESM3, EC-Earth3-Veg-L...
experiment_id                                        [historical]
member_id         [r1i2p2f1, r16i1p1f2, r15i1p1f2, r11i1p1f2, r1...
table_id          [Amon, day, 3hr, 6hrPlev, ImonGre, AERhr, 6hrP...
variable_id                                                [tas]
grid_label                           [gn, gr, grg, gr1, gra, gr2]
 type: object
```

This will populate an underlying pandas dataframe with the search results. The columns of that dataframe and unique values are presented . This exposes more of the control vocabulary for CMIP6. We have already explored `variable_id` and `experiment_id`. Now we explain more of the control vocabulary emphasizing what we find to be the more useful facets.

- `source_id` - The identifier of the model. We use the term *source* instead of *model* in an attempt to make the control vocabulary more general and in the future unify vocabularies among projects. Each model or model version will have a unique string identifying which model and/or configuration was run, which can be browsed.
- `member_id` - The label for the variant of the model run (also known as `variant_label`). The precise meaning of these labels is specific to each model group. For CMIP6 these take the form `r...i...p...f...` where integers after each character reflect a separate run. Usually (but not with all models) the main result will be `r1i1p1f1`.
  - `r` stands for the *realization*. Models can be run with small pertubations of the initial conditions to produce an ensemble. Model runs with the same `r` number started with the same initial conditions.
  - `i` stands for the *initialization*. Models use different methods to spin up their states into quasi-equilibrium. This integer reflects the method that was used by the model.
  - `p` stands for the *physics*. Modern models have many configuration options and while most submit results in a single configuration, this designation provides a method to distinguish among them if desired.
  - `f` stands for the *forcing*. When multiple methods for forcing an experiment are possible, this label distinguishes among them.
- `table_id` - Variables are organized into what CMIP refers to as tables. This tends to be a juxtaposition of a problem realm (`A` for atmosphere, `O` for ocean) along with time frequency (`mon` for month, `day` for day). Note that a variable can exist in several tables. In our search we see that there is `day` temperature data as well as monthly `Amon`.

Skip to content latest

Downloading Data

We will refine our search to select a single model `CanESM5`, variant `r1i1p1f1`, and table `Amon`.

```
cat.search(
    variable_id="tas",
    experiment_id="historical",
    source_id="CanESM5",
    member_id="r1i1p1f1",
    table_id="Amon"
)
```

```
Searching indices: 100%                                    2/2 [     21.18index/s]
```

```
Summary information for 1 results:
mip_era                [CMIP6]
activity_drs            [CMIP]
institution_id         [CCCma]
source_id             [CanESM5]
experiment_id     [historical]
member_id           [r1i1p1f1]
table_id                [Amon]
variable_id              [tas]
grid_label                [gn]
dtype: object
```

Once your search has been sufficiently narrowed, you may download into a dictionary of xarray datasets.

Skip to content

```
lsd = cat.to_dataset_dict()
```

latest

```
Get file information: 100%                                 2/2 [     14.70index/s]
```

```
Downloading 52.4 [Mb]...
```

```
Adding cell measures: 100%                    1/1 [    2.04dataset/s]
```

Note that you do not need to explicitly search for cell measures such as `areacella`. These will be included automatically. The files are downloaded locally to a cache directory which mirrors the directory structure of the remote storate. So while the above code is how you download data, it is also how you load it into memory for your analysis scripts. There is no need to handle files in your working directory or write complicated code to load them into memory.

Plotting

In this example, we will just take a temporal mean and plot the result using matplotlib.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 4), tight_layout=True)
ds = dsd["tas"]["tas"].mean(dim="time") - 273.15  # to [C]
ds.plot(ax=ax, cmap="bwr", vmin=-40, vmax=40, cbar_kwargs={"label": "tas [C]"})
```

```
Matplotlib is building the font cache; this may take a moment.
```

```
<matplotlib.collections.QuadMesh at 0x7dc4b83c94c0>
```